

Addestra il tuo LLM a pensare con GRPO

ora anche su piccole GPU!

Michele Sponsale

2025-04-12

- Un **Large Language Model (LLM)** è una funzione non deterministica che assegna a ogni elemento nel proprio vocabolario (**token**) la **probabilità** di essere il successivo nella sequenza ricevuta in ingresso.
- Le prestazioni dei LLM nella risoluzione di molti problemi migliorano quando generano dei passaggi intermedi (***Chain of Thought***) per arrivare a una soluzione (Wei, 2022; Kojima et al., 2022).

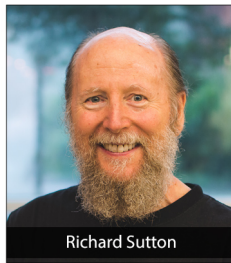
- Per molto tempo addestrare un *reasoning LLM*, ossia in grado di produrre **CoT** di qualità in automatico, è stato difficile.
- **DeepSeek-R1** (DeepSeek-AI, 22 Gennaio 2025) è il secondo frontier reasoning LLM dopo **OpenAI o1**, ed il primo (quasi) open source.
- L'ingrediente segreto di **DeepSeek-R1** è **Group Relative Policy Optimization (GRPO)**.

- GRPO è un algoritmo di **Reinforcement Learning**, un *gradient policy method* introdotto da DeepSeek (Shao et al., 2024).
- È molto efficiente (non tipico nel RL 😞)
- Ha portato a una "esplosione cambriana" di reasoning models!

Questo talk è suddiviso in 3 parti

- Capire la teoria dietro **GRPO**
- Esempio pratico di addestramento con **GRPO** usando *trl* e *Unsloth*
- Topic avanzati

- Il **reinforcement learning** è stato proposto inizialmente da Alan Turing (Turing, 1948) ispirandosi a una forma di apprendimento biologico.
- La formalizzazione corrente, di Sutton e Barto (1980), è valsa loro il Turing Award il 5 Marzo 2025!



Nel *Reinforcement Learning* vogliamo imparare una funzione avendo a disposizione:

- 🍷 un campione di suoi input, ma non dei suoi output (a differenza del *supervised learning*).
- 🍪 una funzione di ricompensa (reward), scelta arbitrariamente da noi, che valuta quanto è buono l'output del nostro modello in risposta agli ingressi che gli forniamo



Robot che impara a camminare: non cadere e spostarsi.



Scrivere una lettera: non fare errori grammaticali.



Risolvere problemi matematici: vicinanza alla soluzione corretta.



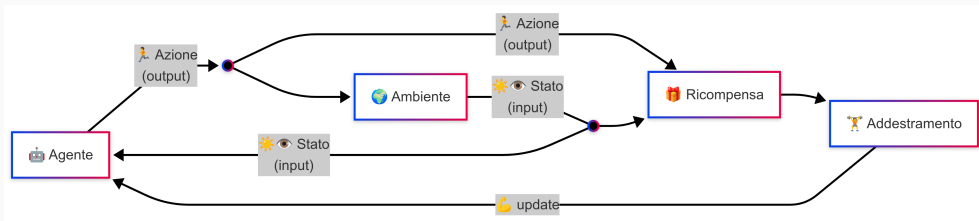
Risolvere un problema NP: la soluzione proposta viene verificata.



Organismo biologico: provare piacere ed evitare il dolore.

Cos'è il Reinforcement Learning?

9



L'obiettivo di addestramento del **RL** è massimizzare il valore atteso della ricompensa del nostro modello.

In molti casi ottimizzarla direttamente è praticamente intrattabile e bisogna introdurre strumenti aggiuntivi.

GRPO fa parte dei metodi di tipo *policy gradient*.

Nei policy gradient facciamo la **salita del gradiente** su una funzione $\mathcal{J}$ detta **obiettivo surrogato** o **performance**.¹

Descrivere un metodo policy gradient si riduce ad indicare il suo obiettivo surrogato $\mathcal{J}$.

¹Sostituiscono discesa del gradiente e loss del supervised learning.

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right),$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1,$$

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right),$$

$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1,$$



The work is mysterious and important.

GRPO è una approssimazione stocastica di **TRPO**, *Trust Region Policy Optimization* (Schulman et al., 2015).

👨🔬: in teoria il reward di **TRPO** cresce² monotonicamente durante l'addestramento.

Per alcune classi di problemi **TRPO** raggiunge sicuramente l'ottimo globale.

²O meglio, non decresce.



Il **vantaggio** $A(input, output)$ ci dice quanto migliore è la ricompensa dell'output corrente rispetto alla ricompensa delle altre coppie (input, output). Formalmente

$$A^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s, A_0 = a \right] - \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right]$$

Il **vantaggio** ci guida verso modelli con ricompensa più alta.

La policy di un modello è la probabilità che questo ci risponda con un certo output a partire da un certo input.

$$\text{policy}(w, x, y) = \Pr(y = \text{model}_w(x))$$

In modelli generativi come gli LLM calcolare la policy è praticamente gratuito:

- **modello** e **policy** quasi coincidono.

Infatti un LLM, insieme al token successivo generato, restituisce anche la probabilità con cui lo genera.

$$\mathcal{J}_{\text{TRPO}}(\text{weight}) = \mathbb{E} \left[\frac{\text{policy}(\text{weight}, -)}{\text{policy}(\text{weight}_{\text{old}}, -)} \cdot A(-) \right]$$

soggetta al vincolo:

$$\mathbb{E}[\mathbb{D}_{KL}(w, w_{\text{old}})] \leq \varepsilon$$

ossia $\mathcal{J}_{\text{TRPO}}$ è il valore atteso del tasso di variazione (cambiamento relativo) della probabilità che il modello risponda con un certo output pesata dal vantaggio che ha quell'output.

$$\mathbb{E}[\mathbb{D}_{KL}(w, w_{\text{old}})] \leq \varepsilon$$

limita la divergenza di Kullback-Leibler, cioè

$$\mathbb{D}_{KL}(P \parallel Q) = \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{q(x)} - \sum_{x \in \mathcal{X}} p(x) \log \frac{1}{p(x)} \quad (1)$$

$$= \mathbb{H}(P, Q) - \mathbb{H}(P, P) \quad (2)$$

ossia limita il cambiamento di contenuto informativo del modello **durante un singolo passo di addestramento**.

Lavorare con l'ottimizzazione vincolata è sconveniente.

In **GRPO** trasformiamo quel vincolo in una penalità:³

$$\mathcal{J}_{\text{TRPO}}(\text{weight}) = \mathbb{E} \left[\frac{\text{policy}(\text{weight}, -)}{\text{policy}(\text{weight}_{\text{old}}, -)} \cdot A(-) - \beta \mathbb{D}_{KL}(w, w_{\text{old}}) \right]$$

³Usando i moltiplicatori di Lagrange.

Calcolare senza approssimazioni la divergenza KL è difficile. In **GRPO** viene stimata come

$$\hat{\mathbb{D}}_{KL}(a, b) = \frac{a}{b} - \log \frac{a}{b} - 1$$

una approssimazione proposta in (Schulman, 2020), veloce e priva di bias.

Problema: I valori attesi in TRPO sono troppo costosi.

Idea: Possiamo sostituirli con una stima Monte-Carlo. Applicarla però ai vantaggi A è non banale.

Soluzione: a ogni passo di addestramento, ripetiamo G volte l'esecuzione del modello (inferenza) su un input fissato. Approssimiamo A e $\mathbb{D}_{KL}$ solo con gli output di quel gruppo di inferenze ripetute.

Il vantaggio approssimato $\hat{A}$ di o_i è lo z -score della sua ricompensa nelle inferenze di addestramento

$$\hat{A}(x, o_i) = \frac{\text{reward}(x, o_i) - \mu[\text{reward}(x, _)]}{\text{std}[\text{reward}(x, _)]}$$

$$\begin{aligned}\mathcal{J}_{GRPO}(\theta) &= \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \\ &\quad \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left(\frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \epsilon, 1 + \epsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \\ \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) &= \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1,\end{aligned}$$

Il $\min \left(\frac{\pi_{\theta}}{\pi_{\theta_{old}}} A_i, \text{clip} \left(\frac{\pi_{\theta}}{\pi_{\theta_{old}}}, 1 - \epsilon, 1 + \epsilon \right) A_i \right)$ serve solo a rimuovere valori campionati estremi del vantaggio surrogato $\frac{\pi_{\theta}}{\pi_{\theta_{old}}} \hat{A}_i$, limitandoli in un intervallo accettabile e stabilizzando l'addestramento.

È ~~rubato~~ tratto da **PPO**, (Schulman et al., 2017), un'altra approssimazione di **TRPO**.

1. Ripeto G volte l'inferenza col modello corrente su un input fissato
2. Calcolo i vantaggi approssimati di ogni output con lo z-score della sua ricompensa
3. Li stabilizzo usando clip (da PPO).
4. Aggiungo la penalità KL approssimata
5. Stimo il valore atteso facendo la media sul gruppo di inferenze
6. uso la funzione così ottenuta con un ottimizzatore (adamW, muon, etc.)

Colli di bottiglia: G inferenze (parallelizzabili). Tutto il resto è semplice aritmetica e un singolo passo di salita del gradiente (o comunque di ottimizzatore).



Unslloth è una libreria che permette di addestrare Transformers based model (come gli llm) con un risparmio di memoria fino al 90%.



Approssimativamente, usando le giuste impostazioni, addestrare con GRPO un LLM richiede circa 1GB di VRAM per miliardo di parametri usando Unslow.

```
[ ] 1  !pip install unsloth vllm
    2
    3  from unsloth import FastLanguageModel, PatchFastRL
    4  PatchFastRL("GRP0", FastLanguageModel)
    5
    6  from unsloth import is_bfloat16_supported
```



```
1  import torch
2  max_seq_length = 1024 # Can increase for longer reasoning traces
3  lora_rank = 64 # Larger rank = smarter, but slower
4
5  model, tokenizer = FastLanguageModel.from_pretrained(
6      model_name = "Qwen/Qwen2.5-3B-Instruct",
7      max_seq_length = max_seq_length,
8      load_in_4bit = True, # False for LoRA 16bit
9      fast_inference = True, # Enable vLLM fast inference
10     max_lora_rank = lora_rank,
11     gpu_memory_utilization = 0.5, # Reduce if out of memory
12 )
```

Non c'è bisogno di imparare nuove librerie: **Unsloth** ottimizza dietro le quinte librerie standard come transformers e trl di HuggingFace!

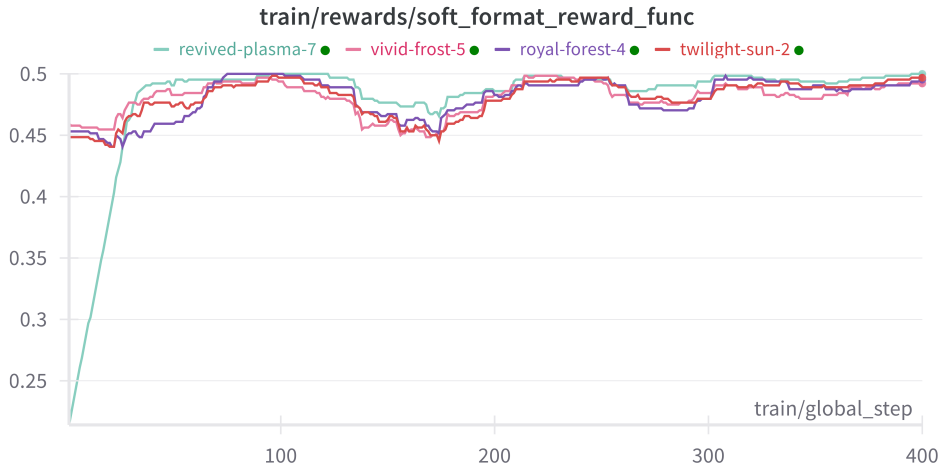
```
[ ] 1  trainer = GRPOTrainer(  
2      model = model,  
3      processing_class = tokenizer,  
4      reward_funcs = [  
5          xmlcount_reward_func,  
6          soft_format_reward_func,  
7          strict_format_reward_func,  
8          int_reward_func,  
9          correctness_reward_func,  
10     ],  
11     args = training_args,  
12     train_dataset = dataset,  
13 )  
14 trainer.train()
```

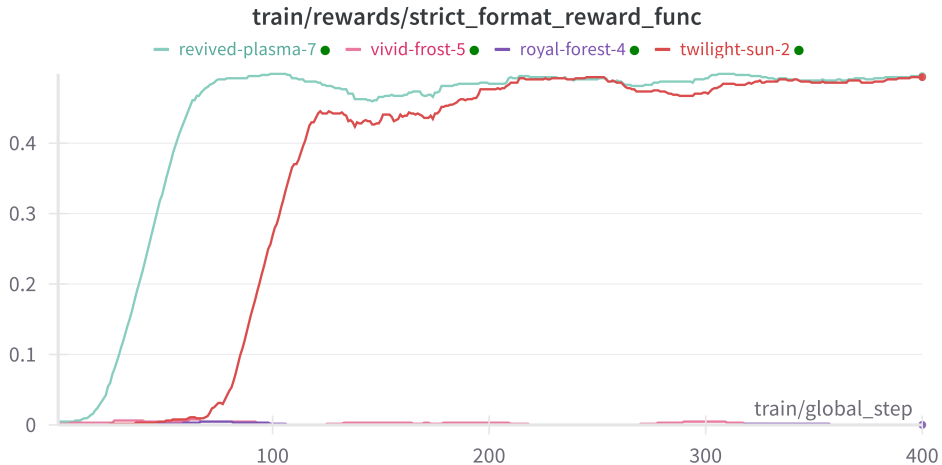
Il parametro di addestramento `reward_funcs` accetta una lista di ricompense. La ricompensa effettiva sarà la somma di tutte queste. Usando una lista le potrò esaminare una a una, diversamente da se le si sommassero a mano.

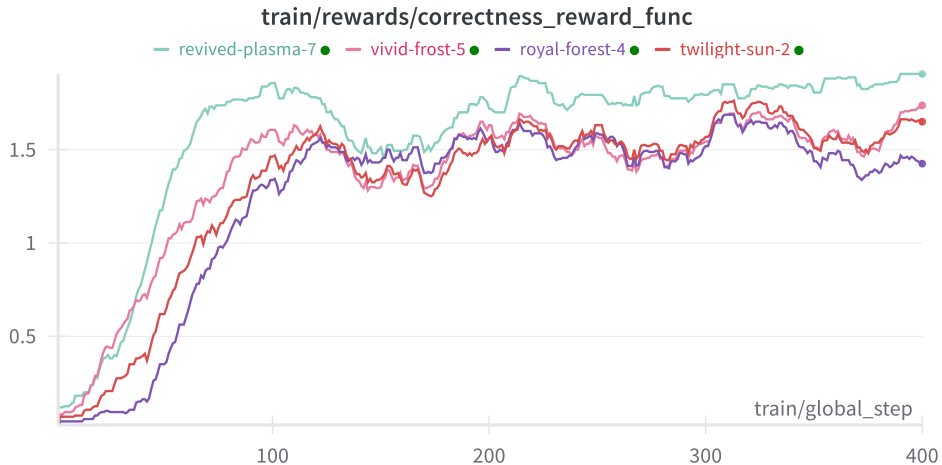
```
reward_funcs = [  
    xmlcount_reward_func,  
    soft_format_reward_func,  
    strict_format_reward_func,  
    int_reward_func,  
    correctness_reward_func,  
],
```

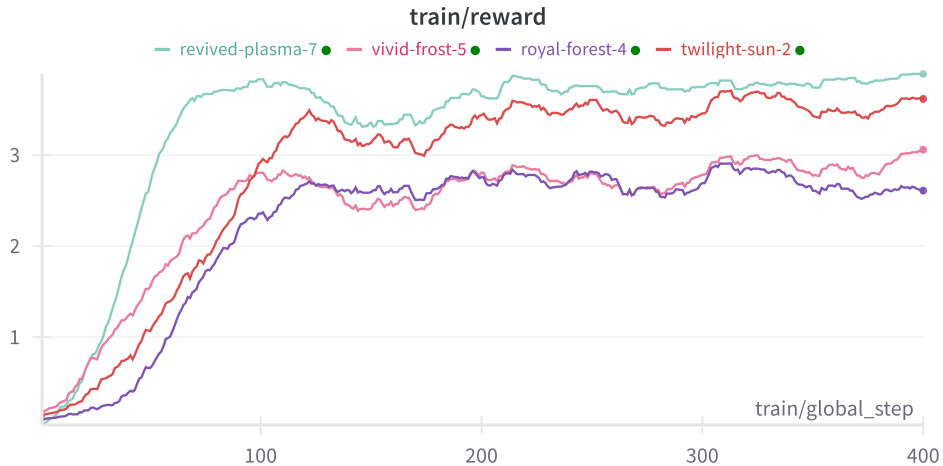

Normalmente possiamo vedere ottimi risultati centinaia di step di training. Con Unsloth, su GPU T4 da 16GB VRAM (Colab), un 3B LLM impiega 2-3 ore per qualche centinaio di passi di GRPO.

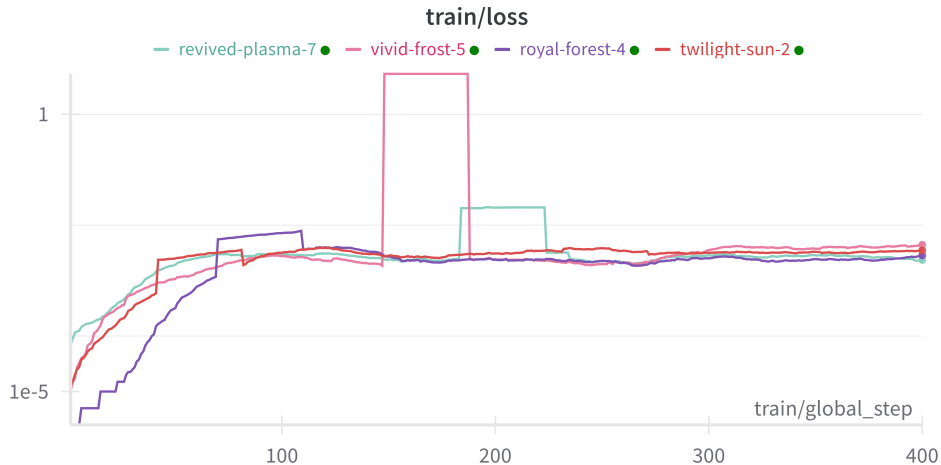
Per esempio questo è quanto serve per passare da performance nell'ordine del 10-20% di correttezza su gsm8k all'80-90%.











Consiglio: dai priorità alla ricompensa e non l'obiettivo surrogato (qui:

- Il **curriculum learning** consiste nell'addestrare un modello in un ordine specifico per migliorare l'apprendimento.
- Di solito si addestra in ordine di difficoltà.
- È simile ai curricula scolastici: gli esseri umani imparano compiti più semplici prima di essere in grado di imparare quelli più complessi.

Stimare la difficoltà di un problema o di sue istanze per il curriculum learning esplicito spesso è difficile o costoso.

Avere più ricompense permette di fare curriculum learning implicito: inizialmente il modello sarà ricompensato per completare compiti più semplici (e.g., seguire una formattazione di risposta) e successivamente per compiti più difficili.

Se il modello ha fatto qualche passo di addestramento supervisionato con esempi di soluzioni di un compito, può convergere più velocemente con GRPO (hot start).

In linea di principio però questo non è necessario se il modello di partenza è sufficientemente sovraparametrizzato.

È meglio usare ricompense obiettive (verificabili), che non dipendono da valutazioni approssimate, da giudizi di esseri umani (**RLHF**) o di LLM (**LLM as a judge**): c'è il rischio che il modello impari a raggirarci piuttosto che a imparare un compito se farlo è più difficile!

In alcuni casi particolari è però accettabile.

Recentemente è stato proposto in (Liu, 2025; Dr. GRPO) che le implementazioni tipiche di GRPO introducano due bias, di lunghezza e difficoltà. Mitigazioni sono già state adottate in **trl**, senza penalizzazioni significative.

Potete trovare queste slides e altro materiale a
<https://github.com/LudWittg/Devfest-2025-Pisa>.



GRAZIE!